

Backend ຕິດຕາມການຊຶກຜ້າແບບ Real-Time

ເອກະສານໂຄງສ້າງລະບົບ · ARCH-001-LO · ກຽມ 2026-09-08 · ສະຖານະ: ຮ່າງ — ລິខັດກວດສອບ Stack: **NestJS 11**
· **Prisma ORM** · **PostgreSQL 16** · **Redis 7** · **Socket.IO** · **BullMQ**

ລະບົບຕິດຕາມສະຖານະການຊຶກຜ້າແບບ real-time ສໍາລັບຮ້ານທີ່ມີພະນັກງານໃຫ້ບໍລິການ — ລູກຄ້າສາມາດເຂົ້າເບິ່ງສະຖານະການຊຶກຜ້າ, ພະນັກງານ
ດໍາເນີນການຊຶກ → ຫາກ → ພັບ, ແລະ ທຸກການປ່ຽນສະຖານະຈະໄປເຖິງແອັບຂອງລູກຄ້າໃນທັນທີ.

ສາລະບານ

- ພາບລວມ ແລະ ຜູ້ນຳໃຊ້
- ໂຄງສ້າງອົງປະກອບ
- ການໄຫຼວຽນຂໍ້ມູນ
- ການອອກແບບ API
- ໂຄງສ້າງຖານຂໍ້ມູນ
- ຜູ້ດຶກສາດການແຄຊ
- ການພິສູດຢືນຢັນຕົວຕົນ ແລະ ການອະນຸຍາດ
- ການຕິດຕາມກວດສອບ
- ຄວາມປອດໄພ
- ແຜນການຈັດຕັ້ງປະຕິບັດ
- ທາງເລືອກເຕັກໂນໂລຊີ
- ຫັກສະ Agent ສໍາລັບ Stack ນີ້

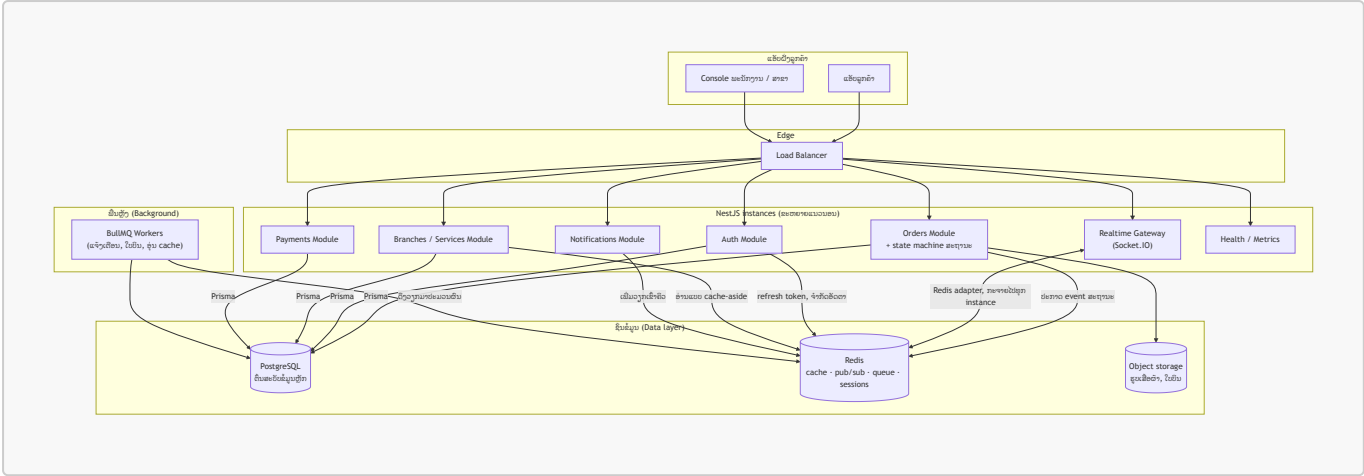
1. ພາບລວມ ແລະ ຜູ້ນຳໃຊ້

ນີ້ແມ່ນຮູບແບບ "ບໍລິການເຕັມຮູບແບບ" ບໍ່ແມ່ນ self-service — ລູກຄ້າມອບເສື້ອຜ້າໃຫ້ ແລະ ບໍ່ໄດ້ແຕ່ງຕັ້ງເຄື່ອງຈັກເລີຍ. ຫ້າທີ່ຂອງລະບົບ
ແມ່ນເຮັດໃຫ້ການມອບໝາຍນີ້ໜ້າເຊື່ອຖື — ລູກຄ້າທຸກຄົນເຫັນຢ່າງແນ່ນອນວ່າຄໍາສັ່ງຊຶກຂອງເຂົາເຈົ້າຢູ່ໃສ, ແບບ real-time, ໂດຍບໍ່ຕ້ອງໂທຫາ
ຮ້ານ.

Role	ໜ້າວຽກ
CUSTOMER (ລູກຄ້າ)	ວາງ ແລະ ຕິດຕາມຄໍາສັ່ງຊຶກ — ນັດວັນຮັບເສື້ອຜ້າ, ເບິ່ງສະຖານະແບບ real-time, ຈ່າຍເງິນ, ໃຫ້ຄະແນນ
STAFF (ພະນັກງານ)	ດໍາເນີນການເສື້ອຜ້າ — ຄັດແຍກ, ຊັງນ້ຳໜັກ, ປ່ຽນສະຖານະຢູ່ແຕ່ລະຈຸດ, ຕິດຮູບພາບກໍລະນີຂັດແຍ່ງ
DRIVER (ຄົນຂັບ)	ຮັບ-ສົ່ງ — ບົດບາດທາງເລືອກ; ອັບເດດສະຖານະຕອນຮັບ ແລະ ຕອນສົ່ງເຖິງ
MANAGER (ຜູ້ຈັດການ)	ບໍລິຫານສາຂາ — ມອບໝາຍພະນັກງານ, ຕິດຕາມປະສິດທິພາບ, ຈັດການຄົນເງິນ ແລະ ຂໍ້ຂັດແຍ່ງ

2. ໂຄງສ້າງອົງປະກອບ

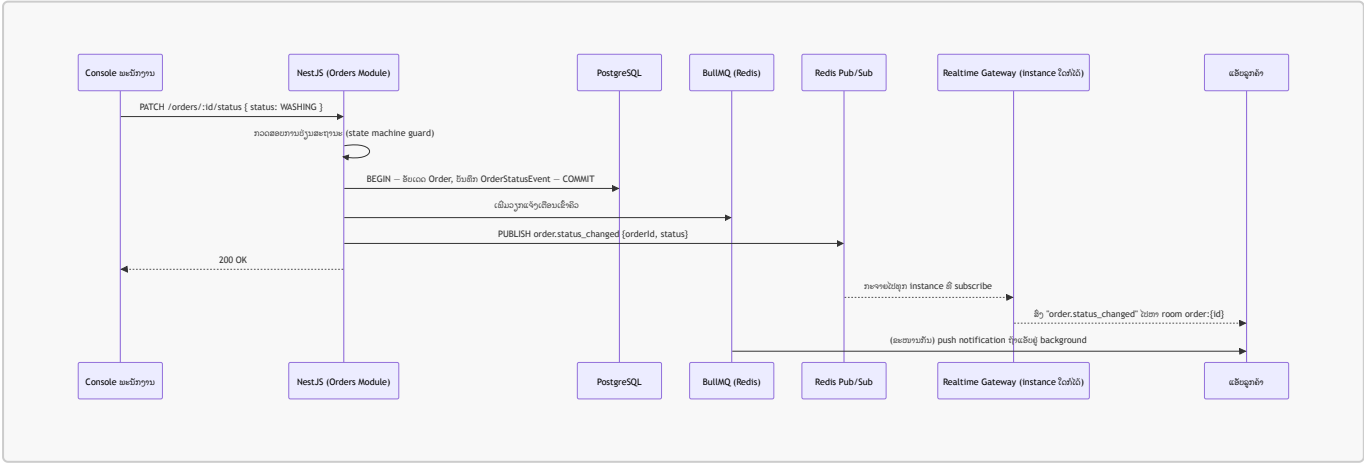
Modular monolith ໃນ NestJS: ໜຶ່ງ deployable, ຂອບເຂດ module ທີ່ເຄັ່ງຄັດ, ດັ່ງນັ້ນສ່ວນຕ່າງໆ (ເຊັ່ນ Payments, Realtime) ຈຶ່ງແຍກອອກເປັນ service ຕ່າງຫາກພາຍຫຼັງໄດ້ ໂດຍບໍ່ຕ້ອງຂຽນໃໝ່ທັງໝົດ.



Module	ຮັບຜິດຊອບ
AuthModule	ເຂົ້າສູ່ລະບົບ, ຫຼຸດຜູ້ໃຊ້, ຫຼຸດຜູ້ໃຊ້ refresh token, guards, RBAC, OTP ທາງໂທລະສັບ (ທາງເລືອກ)
UsersModule	ໂປຣໄຟລ໌ ລູກຄ້າ ແລະ ພະນັກງານ, ທີ່ຢູ່, device token
BranchesModule	ສະຖານທີ່ຮ້ານ, ເວລາເປີດ-ປິດ, ຄວາມສາມາດຮັບອໍເດີ
ServicesModule	ບັນຊີລາຄາ — ຊັກ/ຫາງ/ອັດ, ຄິດເປັນກິໂລ ຫຼື ຕໍ່ຕົວ
OrdersModule	ວົງຈອນຊີວິດຄໍາສັ່ງຊື້, state machine, ປະຫວັດສະຖານະ (ຕົ້ນສະບັບຄວາມຈິງ)
RealtimeModule	Socket.IO gateway, ການເຂົ້າຮ່ວມ room, ການເຊື່ອມຕໍ່ Redis adapter
PaymentsModule	payment intent, webhook ຂອງຜູ້ໃຫ້ບໍລິການ, idempotency
NotificationsModule	ສົ່ງແຈ້ງເຕືອນ push / SMS / email ຜ່ານ BullMQ producer
PrismaModule / RedisModule	client ແບບ global, injectable — ໜຶ່ງ connection pool ຕໍ່ instance
HealthModule	liveness/readiness probe, endpoint metrics ຂອງ Prometheus

3. ການໄຫຼວຽນຂໍ້ມູນ — ການປ່ຽນສະຖານະແບບຄົບວົງຈອນ

ຈຸດທີ່ຕ້ອງໃຫ້ຄວາມຮູ້ສຶກວ່າໄວທັນທີ: ພະນັກງານສະແກນຄໍາສັ່ງຊື້ໃຫ້ກ້າວໜ້າໜຶ່ງຂັ້ນ ແລ້ວແອັບຂອງລູກຄ້າອັບເດດເອງໂດຍບໍ່ຕ້ອງ refresh.



ການສົ່ງຜ່ານ Redis pub/sub ນີ້ແມ່ນສິ່ງທີ່ເຮັດໃຫ້ລະບົບເຮັດວຽກໄດ້ໃນຫຼາຍ instance ຂອງ NestJS: instance ທີ່ຖື socket ຂອງລູກຄ້າ ບໍ່ຈຳເປັນຕ້ອງເປັນ instance ດຽວກັນກັບທີ່ຮັບ API request.

4. ການອອກແບບ API

REST ສໍາລັບຄໍາສັ່ງ ແລະ ການອ່ານຂໍ້ມູນ, ມີເວີຊັນຢູ່ພາຍໃຕ້ `/api/v1`; ໜຶ່ງ Socket.IO namespace ສໍາລັບທຸກຢ່າງທີ່ຕ້ອງມາເຖິງໂດຍບໍ່ໄດ້ຮ້ອງຂໍ.

Endpoint	ໃຜ	ຈຸດປະສົງ
POST /auth/login	ທຸກຄົນ	ເຂົ້າສູ່ລະບົບດ້ວຍລະຫັດຜ່ານ ຫຼື OTP, ອອກ access + refresh token
POST /auth/refresh	ທຸກຄົນ	ໝູນວຽນ refresh token (ເກັບໄວ້ໃນ Redis, ອີກເລິກໄດ້ທັນທີ)
GET /branches/:id/services	ທຸກຄົນ	ບັນຊີລາຄາແບບສິດຂອງສາຂາ (cache ໄວ້)
POST /orders	ລູກຄ້າ, ພະນັກງານ	ສ້າງຄໍາສັ່ງຊື້ — ຈອງຜ່ານແອັບ ຫຼື ພະນັກງານປ້ອນລູກຄ້າ walk-in
GET /orders/:id	ເຈົ້າຂອງ, ພະນັກງານ	ລາຍລະອຽດຄໍາສັ່ງຊື້ + ສະຖານະປັດຈຸບັນ
GET /orders/:id/history	ເຈົ້າຂອງ, ພະນັກງານ	ໄຫມໄລນ໌ສະຖານະທັງໝົດ, ຈາກ <code>OrderStatusEvent</code>
PATCH /orders/:id/status	ພະນັກງານ, ຄົນຂັບ	ເລື່ອນ state machine ໄປໜຶ່ງການປ່ຽນທີ່ຖືກຕ້ອງ
POST /orders/:id/items/:itemId/photo	ພະນັກງານ	ຜິດຮູບຫຼັກຖານເສື້ອຜ້າ (pre-signed S3 upload)
POST /orders/:id/payments/intent	ລູກຄ້າ	ສ້າງ payment intent, ມີ idempotency key
POST /payments/webhook	ຜູ້ໃຫ້ບໍລິການ	ການຢືນຢັນການຈ່າຍເງິນແບບ server-to-server ທີ່ເຊັນແລ້ວ
WS /realtime	ລູກຄ້າ, ພະນັກງານ	Room: <code>customer:{id}</code> , <code>branch:{id}:staff</code>
GET /health · /metrics	ຝ່າຍປະຕິບັດງານ	liveness/readiness, ຈຸດໃຫ້ Prometheus scrape

ສະຖານະຄໍາສັ່ງຊື້ — State Machine

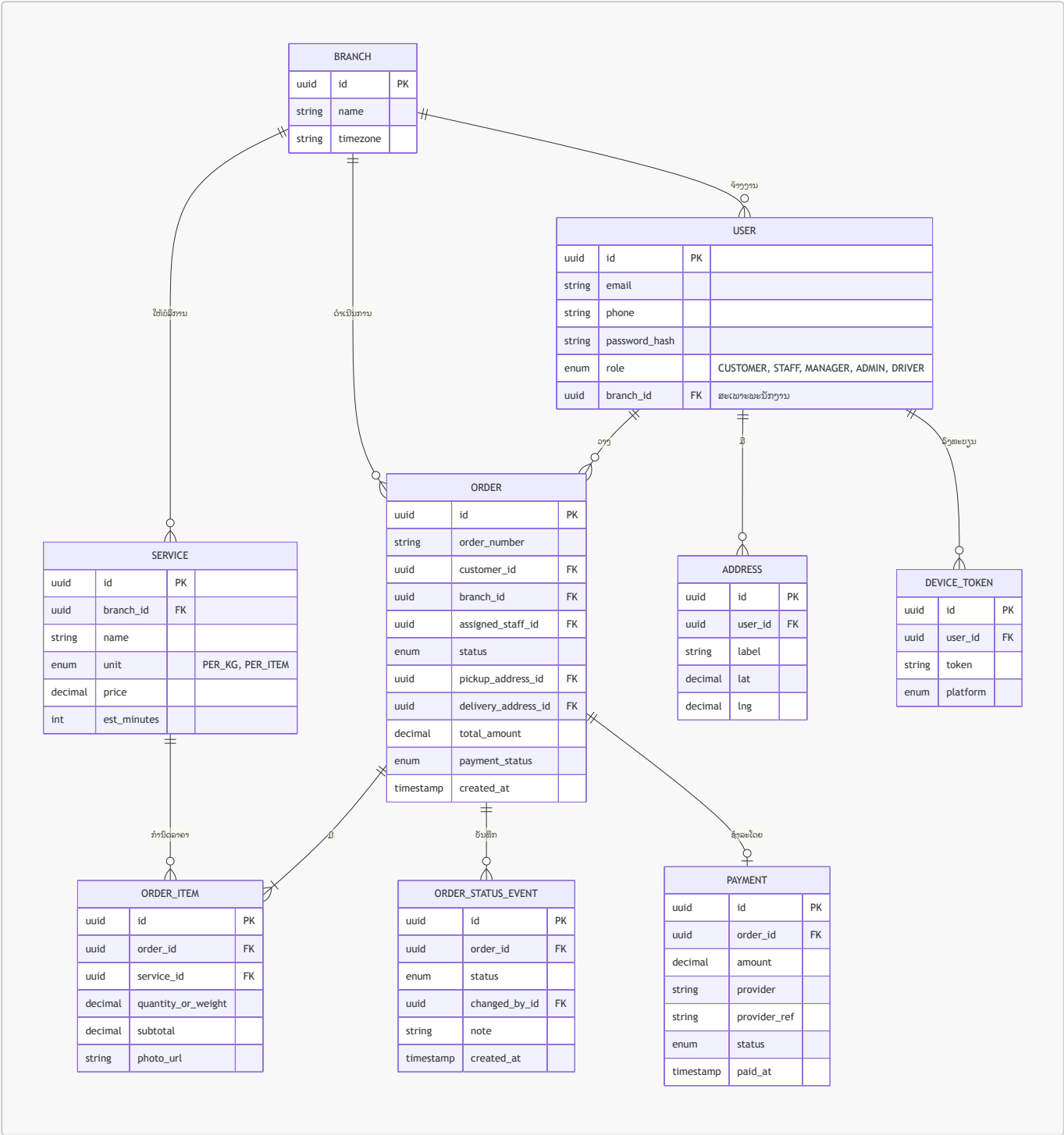
ບັງຄັບໃຊ້ຢູ່ຝັ່ງ server ບ່ອນດຽວ; API ຈະປະຕິເສດການປ່ຽນສະຖານະໃດໆທີ່ບໍ່ຢູ່ໃນລໍາດັບນີ້.

PLACED → PICKED_UP → AT_FACILITY → WASHING → DRYING → FOLDING
→ QUALITY_CHECK → READY → OUT_FOR_DELIVERY → DELIVERED

(ຈາກສະຖານະໃດໜຶ່ງກ່ອນ WASHING) → CANCELLED

5. ໂຄງສ້າງຖານຂໍ້ມູນ

OrderStatusEvent ເປັນແບບ append-only ແລະ ເປັນຕົ້ນສະບັບຄວາມຈິງທີ່ແທ້ຈິງ — **Order.status** ເປັນ "ຄ່າປັດຈຸບັນ" ແບບ denormalized ທີ່ຖືກຮັກສາໃຫ້ກົງກັນພາຍໃນ transaction ດຽວກັນ, ດັ່ງນັ້ນສະຖານະສົດ ແລະ ບັນທຶກກວດສອບຈຶ່ງບໍ່ມີວັນຂັດແຍ່ງກັນ.



ໝາຍເຫດເລື່ອງ Index

- **Order(branch_id, status, created_at)** — query ຫຼັກຂອງ dashboard ສາຂາ, "ຕອນນີ້ມີຫຍັງແດ່ທີ່ກຳລັງດຳເນີນການຢູ່"
- **Order(customer_id, created_at desc)** — ປະຫວັດຄຳສັ່ງຊື້ຂອງລູກຄ້າ
- **OrderStatusEvent(order_id, created_at)** — ການອ່ານ timeline, ຮຽງລຳດັບສະເໝີ
- **Order.order_number** unique — ເລກທີ່ອ່ານໄດ້, ພິມຢູ່ປ້າຍເສື້ອຜ້າ (claim tag)

ໝາຍເຫດ: scaffold ປັດຈຸບັນຕັ້ງຄ່າ Prisma ດ້ວຍ `@prisma/adapters-better-sqlite3` — ໃຫ້ປ່ຽນ provider ຂອງ datasource ເປັນ `postgresql` ແລະ ປ່ຽນມາໃຊ້ `@prisma/adapters-pg` ກ່ອນສ້າງ migration ທໍາອິດ.

6. ຍຸດທະສາດການແຄຊ

Redis ຮັບຜິດຊອບ 4 ໜ້າວຽກທີ່ແຕກຕ່າງກັນຢູ່ນີ້ — ການປະປົນມັນເຂົ້າກັນເປັນ client/config ດຽວແມ່ນວິທີທີ່ໄປທີ່ສຸດທີ່ລະບົບແບບນີ້ຈະ ຍາກຕໍ່ການເຂົ້າໃຈໃນພາຍຫຼັງ, ດັ່ງນັ້ນແຕ່ລະໜ້າວຽກຈຶ່ງໄດ້ key namespace ຂອງຕົນເອງ.

ການໃຊ້ງານ	ຮູບແບບ	ຮູບແບບ Key
Read-through cache	Cache-aside, TTL 5–15 ນາທີ, ລຶບ cache ທັນທີເມື່ອຂຽນ	<code>branch:{id}:services</code>
Session / refresh token	TTL = ອາຍຸ token; ລຶບເມື່ອ logout ເພື່ອ ຍົກເລີກທັນທີ	<code>session:refresh:{jti}</code>
Realtime fan-out	Pub/sub ຮອງຮັບ Socket.IO Redis adapter ຂ້າມ instance	<code>socket.io#order.status_changed</code>
Job queue	BullMQ — ແຈ້ງເຕືອນ, ໃບບິນ PDF, ຮຸ່ນ cache ຕາມກຳນົດເວລາ	<code>bull:notifications:*</code>
Rate limiting	ຕົວນັບແບບ sliding window ຢູ່ຫຼັງ <code>@nestjs/throttler</code>	<code>throttle:{ip userId}:{route}</code>
Distributed lock	Redlock — ປ້ອງກັນການສ້າງເລກຄຳສັ່ງຊື້ ພ້ອມກັນ	<code>lock:order-number:{branchId}</code>

7. ການພິສູດຢືນຢັນຕົວຕົນ ແລະ ການອະນຸຍາດ

Token

- JWT access token ອາຍຸສັ້ນ (~15 ນາທີ), Passport `JwtStrategy`
- Refresh token ໝູນວຽນ, hash ໄວ້ໃນ Redis ດ້ວຍ TTL ທີ່ກົງກັນ
- Password hash ດ້ວຍ `argon2`, ບໍ່ແມ່ນ bcrypt
- ທາງເລືອກ: ເຂົ້າສູ່ລະບົບດ້ວຍ OTP ທາງໂທລະສັບສໍາລັບລູກຄ້າ

ການອະນຸຍາດ (Authorization)

- Decorator `@Roles()` + `RolesGuard`: CUSTOMER, STAFF, MANAGER, ADMIN, DRIVER
- Endpoint ຂອງພະນັກງານຖືກຈຳກັດເພີ່ມຕາມ `branch_id` ຂອງຕົນເອງ
- WebSocket handshake ຖື JWT ດຽວກັນ; gateway ກວດສອບກ່ອນເຂົ້າຮ່ວມ room
- Socket ເຂົ້າຮ່ວມພຽງ 2 room: `customer:{id}` ຂອງຕົນເອງ ຫຼື staff room ຂອງສາຂາຕົນເອງ

8. ການຕິດຕາມກວດສອບ

- **Logs** — `nestjs-pino`, JSON ແບບມີໂຄງສ້າງ, correlation ID ຕໍ່ request, ສົ່ງໄປ Loki ຫຼື CloudWatch
- **Metrics** — `@willsoto/nestjs-prometheus` — latency ຂອງ request, ຄວາມເລັກຄົວ, ອັດຕາ cache hit, ຈຳນວນການເຊື່ອມຕໍ່ socket
- **Traces ແລະ Errors** — OpenTelemetry ຂ້າມ API → Prisma → Redis; Sentry ສໍາລັບ exception ພ້ອມ context ຂອງ request

Health check ຜ່ານ [@nestjs/terminus](#) ກວດ Postgres, Redis, ແລະ disk — ເຊື່ອມກັບ readiness probe ຂອງ load balancer ດັ່ງນັ້ນ instance ທີ່ເສຍຈະຖືກດຶງອອກຈາກ rotation, ບໍ່ແມ່ນພຽງແຕ່ log ໄວ້.

9. ຄວາມປອດໄພ

- ValidationPipe ແບບ global ດ້ວຍ [whitelist](#) + [forbidNonWhitelisted](#) — ບໍ່ມີພົວພັນທີ່ໄດ້ປະກາດເຂົ້າມາເຖິງ handler
- Parameterized query ຂອງ Prisma ປິດຊ່ອງ SQL injection; [\\$queryRawUnsafe](#) ຖືກຫ້າມດ້ວຍ lint rule, ບໍ່ແມ່ນພຽງແຕ່ຄໍາແນະນໍາ
- Helmet + CORS allowlist ທີ່ເຄັ່ງຄັດຕໍ່ environment; HTTPS/HSTS ທຸກບ່ອນ
- Idempotency key ຢູ່ endpoint ສ້າງຄໍາສັ່ງຊື້ ແລະ ຈ່າຍເງິນ — request ທີ່ສົ່ງຊ້າຈະບໍ່ສາມາດເກັບເງິນຊ້າ ຫຼື ຈອງຊ້າໄດ້
- ຮູບເສື້ອຜ້າສົ່ງກົງໄປ object storage ຜ່ານ pre-signed URL — API ບໍ່ຕ້ອງ proxy ໄຟລ໌ເລີຍ
- Secret ຢູ່ນອກ repo (gitignore ໄວ້ແລ້ວ) — ໃຊ້ secrets manager ແທ້ໆໃນ staging/production, ບໍ່ແມ່ນໄຟລ໌ [.env](#)
- ເບີໂທ ແລະ ທີ່ຢູ່ຂອງລູກຄ້າແມ່ນຂໍ້ມູນສ່ວນຕົວ — ເຂົ້າລະຫັດຂໍ້ມູນຕອນເກັບ ແລະ ຈໍາກັດການເຂົ້າເຖິງຕາມ role ຖ້າກົດໝາຍປົກປ້ອງຂໍ້ມູນທ້ອງຖິ່ນ (ເຊັ່ນ PDPA) ນໍາໃຊ້
- [OrderStatusEvent](#) ເຮັດໜ້າທີ່ເປັນ audit log ນໍາ; ການກະທໍາສໍາຄັນຂອງ admin (ຄືນເງິນ, ປ່ຽນສະຖານະດ້ວຍມື) ຄວນມີ audit trail ຂອງຕົນເອງ

10. ແຜນການຈັດຕັ້ງປະຕິບັດ

ຈັດລຳດັບເພື່ອໃຫ້ແຕ່ລະໄລຍະຖິ້ມສິ່ງທີ່ສາທິດໄດ້ຈົງໄວ້ — ບໍ່ແມ່ນ abstraction ເຮັດຄ້າງໄວ້ລໍຖ້າໄລຍະຕໍ່ໄປ.

ໄລຍະ	ຫົວຂໍ້	ລາຍລະອຽດ
00	ພື້ນຖານ	ຕັ້ງ Prisma ໃຫ້ຊື້ໄປ PostgreSQL (ປ່ຽນຈາກ adapter sqlite), Docker Compose ສໍາລັບ Postgres + Redis, ກວດສອບ env ດ້ວຍ zod, ໂຄງ CI ພື້ນຖານ
01	ຕົວຕົນ (Identity)	Users, roles, JWT auth + ໝູນວຽນ refresh token ໃນ Redis, guards — ພຽງພໍທີ່ຈະ login ເປັນລູກຄ້າ ແລະ ພະນັກງານໄດ້
02	ໂດເມນຫຼັກ	Branches, Services, Orders, OrderItems, state machine ຂອງສະຖານະ ແລະ ບັນທຶກ OrderStatusEvent — ຍັງບໍ່ມີ realtime, REST ລ້ວນໆ
03	Realtime	Socket.IO gateway + Redis adapter; ເຊື່ອມການປ່ຽນສະຖານະໃຫ້ broadcast — ນີ້ຄືຄຸນສົມບັດທີ່ຜະລິດຕະພັນນີ້ຖືກຕັ້ງຊື່ຕາມ
04	ວຽກແບບ Async	BullMQ ສໍາລັບແຈ້ງເຕືອນ push/SMS, ອອກໃບບິນ, ຮຸ່ນ cache ຕາມກໍານົດເວລາ
05	ການຈ່າຍເງິນ	ເຊື່ອມຕໍ່ຜູ້ໃຫ້ບໍລິການ, ຈັດການ webhook ທີ່ເຊັນແລ້ວ, idempotency key ຢູ່ເສັ້ນທາງການຂຽນ
06	ການສັງເກດການ	Logging, metrics, tracing, health check, Sentry — ກ່ອນມີ load ຫຼາຍ, ບໍ່ແມ່ນຫຼັງເກີດເຫດການ
07	ເສີມຄວາມແຂງແຮງ ແລະ ຂະຫຍາຍ	Rate limiting, ທົດສອບ load ດ້ວຍ k6, read replica ຖ້າຕ້ອງການ, ຂະຫຍາຍ NestJS pool ແບບອັດຕະໂນມັດ

11. ທາງເລືອກເທັກໂນໂລຊີ

ດ້ານ	ທາງເລືອກ	ເຫດຜົນ
Framework	NestJS 11	ກຳນົດມາແລ້ວ — DI ແບບ module ເໝາະກັບຂອບເຂດ module ຂ້າງເທິງ
ORM	Prisma, @prisma/adapters-pg	ກຳນົດມາແລ້ວ — migration ແບບ type-safe; ຕ້ອງປ່ຽນ datasource ຈາກ sqlite
Database	PostgreSQL 16	ກຳນົດມາແລ້ວ — row-level locking ແລະ JSON column ຮອງຮັບໃບບິນ/metadata ໂດຍບໍ່ຕ້ອງມີ store ທີ 2
Cache / broker	Redis 7	ກຳນົດມາແລ້ວ — engine ດຽວຮັບໃຊ້ cache, pub/sub, queue ແລະ rate-limit
Realtime transport	Socket.IO + @socket.io/redis-adapter	ມີ room ແລະ reconnection ພ້ອມໃຊ້; ຂະຫຍາຍຂ້າມ instance ຜ່ານ Redis
Job queue	BullMQ	ໃຊ້ Redis ໂດຍກົງ, ເຊື່ອມກັບ queue module ຂອງ NestJS ໂດຍກົງ
Auth	Passport-JWT + argon2	ເສັ້ນທາງ auth ມາດຕະຖານຂອງ NestJS; argon2 ເປັນຄຳແນະນຳປັດຈຸບັນສໍາລັບ hash password
File storage	S3-compatible (S3 / R2)	Pre-signed upload ຮັກສາຮູບເສື້ອເປັນຕົ້ນໃຫ້ຢູ່ນອກເສັ້ນທາງ request ຂອງ API
Deployment	Docker → ECS/Fargate ຫຼື Kubernetes	API pod ແບບ stateless ຢູ່ຫຼັງ load balancer; Postgres/Redis ຈັດການແຍກຕ່າງຫາກ

12. ທັກສະ Agent ສໍາລັບ Stack ນີ້

ຈາກ [skillsmp.com](#) — ທັກສະທີ່ກົງກັບ stack ນີ້ (NestJS + Prisma + PostgreSQL + Redis) ທັງໝົດມາຈາກຜູ້ພັດທະນາຄົນດຽວ, [affaan-m/ECC](#), ແລະ ອ່ານຄືເປັນຊຸດທີ່ເຂົ້າກັນ ບໍ່ແມ່ນວຽກແຍກແຕ່ລະຊິ້ນ.

ທັກສະ	ຄອບຄຸມ	ຈຸດທີ່ຊ່ວຍໃນນີ້
nestjs-patterns	Module, controller, DTO validation, guard, interceptor	ຮັກສາຂອບເຂດ module ໃນ §2 ໃຫ້ສອດຄ່ອງເມື່ອແອັບໃຫຍ່ຂຶ້ນ
prisma-patterns	ການອອກແບບ schema, ປັບປຸງ query, transaction, pagination	Transaction ຂອງ state machine ສະຖານະ ແລະ pagination ຂອງປະຫວັດຄໍາສັ່ງຊື້
postgres-patterns	Index, ປັບ query, row-level security	Composite index ໃນ §5, ແລະ RLS ຖ້າຕ້ອງແຍກຂໍ້ມູນພະນັກງານ/ລູກຄ້າຢູ່ລະດັບ DB
redis-patterns	Data structure, caching, distributed lock, pub/sub, rate limiting	ກົງກັບທຸກແຖວຂອງຕາຕະລາງ §6 ໂດຍກົງ
database-migrations	ປ່ຽນ schema ແບບ zero-downtime, rollback	Migration ທີ່ປອດໄພເມື່ອຕາຕະລາງ order/payment ມີຂໍ້ມູນຈົງແລ້ວ
documentation-lookup	ດຶງເອກະສານ library ຫຼ້າສຸດຜ່ານ Context7 ແທນທີ່ຈະໃຊ້ training data	ເປັນປະໂຫຍດເມື່ອງຈາກ NestJS/Prisma ອອກເວີຊັນໃໝ່ໄວ

Code ຂອງໜັກສະຖິກເຜີຍແຜ່ໂດຍຊຸມຊົນ — ຄວນອ່ານກ່ອນຕິດຕັ້ງ ຄືກັນກັບ dependency ອື່ນໆ.

Backend ຕິດຕາມການຊັກຮັກ — ເອກະສານໂຄງສ້າງລະບົບ · ກຽມສໍາລັບການທົບທວນພາຍໃນ